

# COMP1521

## TUTORIAL 1

ADMIN → C REVISION → RECURSION → COMPILATION → MIPS INTRO

# ICEBREAKERS!

- NAME
- DEGREE AND YEAR
- BORING FACT ABOUT YOURSELF

# Assessment Structure

- 9 Sets of Weekly Labs (15%)
- 8 Weekly Tests (10%)
  - First is in week 3 (there is also one during flex week)
  - Best 6 of 8 is taken for total test mark
- 2 Assignments (Each 15%)
  - Assignment 1 is typically a game translated from C to MIPS
  - Assignment 2 is written in C and typically involves bitwise operations and files
- Final Exam (45%)
  - Typically, this has a hurdle of at least 40% mark in final

# Getting Full Marks in Labs

- 9 weeks of labs (2 marks each)
  - Normal exercises are worth 1.6
  - Challenge exercises are worth 0.4
- All non-challenge exercises will add up to 14.4/15
- To get 15/15 typically need to complete 2 weeks of challenge exercises
  - Or about 4 challenge exercises total over the term
  - Pick your battles with the challenge exercises some are much more difficult than others!

# Submission

- All work is submitted through **give** using the command line
  - commands are provided for labs/tests/assignments
- Autotests are also provided where applicable (basically everywhere)

○ ○ ○

```
give cs1521 lab01_no_vowels no_vowels.c
```

○ ○ ○

```
1521 autotest no_vowels
```

# Course Website and Forum

Course Website: <https://cgi.cse.unsw.edu.au/~cs1521/25T3>

Course Forum: <https://discourse02.cse.unsw.edu.au/25T3/COMP1521/>

- Join the forum! Tutors are almost always around to help, and answer questions

# Help Sessions

<https://cgi.cse.unsw.edu.au/~cs1521/25T3/help-sessions/>

- Start in **week 3/4** around assignment 1 release
- Swing by for help debugging, lab or assignment help
- Usually, they are in **K17** in **CSEHELP**, or **SEM113**
  - But always check the room on the schedule!

C REVISION!



Write a c program `count_chars.c` that uses `getchar()` to read in characters until the user enters Ctrl-D and then print the total number of characters entered.

Use `man 3 getchar` to get the manual entry

```
○ ○ ○  
  
#include <stdio.h>  
  
int main(void) {  
    int ch;  
    int count = 0;  
  
    while ((ch = getchar()) != EOF) {  
        count += 1;  
    }  
  
    printf("Characters entered = %d\n", count);  
  
}
```

In the following program `print_arguments.c`, what are `argc` and `argv`?

○ ○ ○

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("argc=%d\n", argc);

    for (int i = 0; i < argc; i++) {
        printf("argv[%d]=%s\n", i, argv[i]);
    }

    return 0;
}
```

○ ○ ○

```
./print_arguments I love MIPS
```

**Argc = 4**

**Argv[0] = print\_arguments**

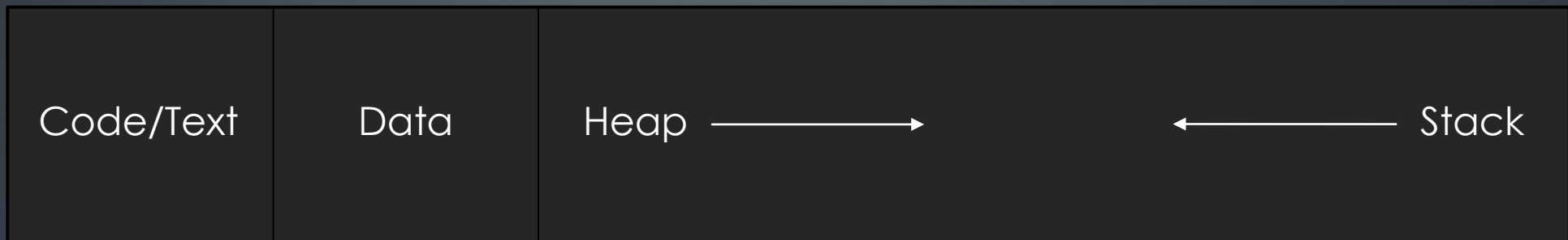
**Argv[1] = I**

**Argv[2] = love**

**Argv[3] = MIPS**

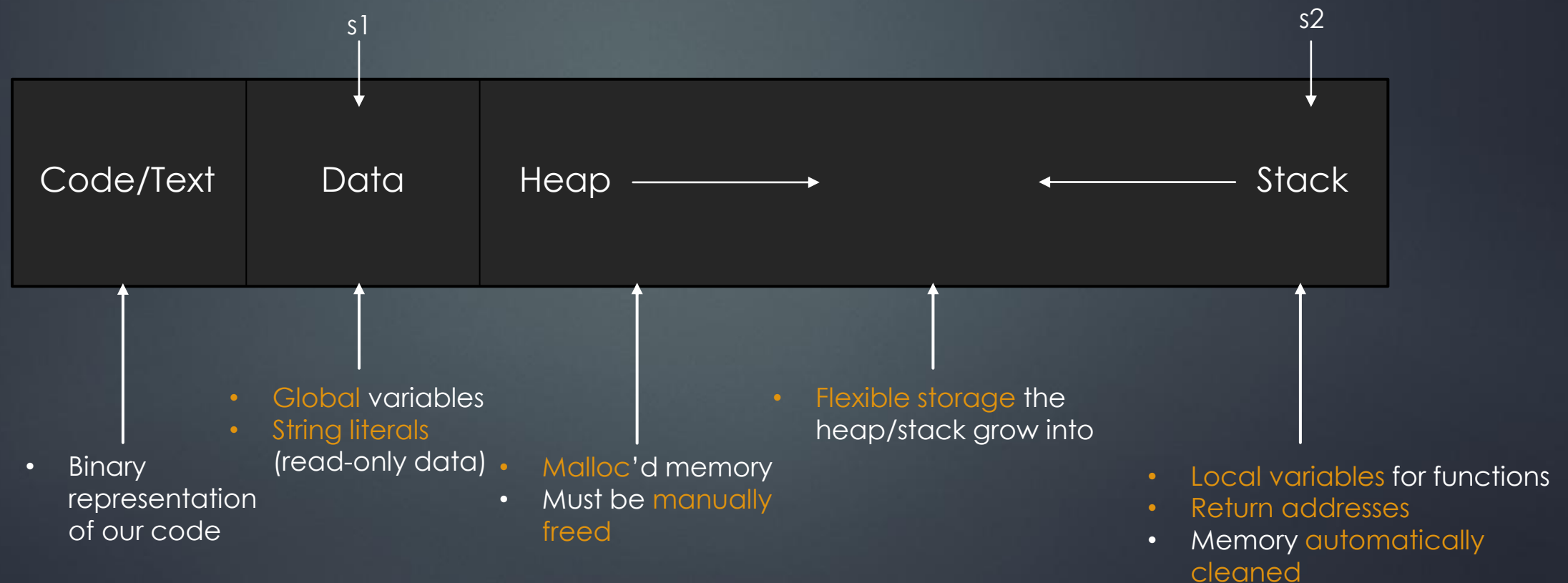
```
ooo  
#include <stdio.h>  
char *s1 = "abc";  
int main(void) {  
    char *s2 = "def";  
    // ...  
}
```

- How do **global variables** differ from **local variables** in functionality?
- What does the **memory layout of a typical program** look like?
- Where are **global variables** located?
- Where are **local variables** located?
- What is a **string literal**? Where are they located?



When our computer executes our code, the operating system spawns a **process**, essentially a task the computer will run.

Each process will be allocated a section of memory by the operating system, which typically resembles the diagram below.



## Example: get\_num\_ptr.c

What's wrong with this code?

Assume we still want `get_num_ptr()` to return a pointer, how can we fix this code?

How did the change made affect each variable's **location** in **memory**?

```
○ ○ ○  
  
#include <stdio.h>  
  
int *get_num_ptr(void);  
  
int main(void) {  
    int *num = get_num_ptr();  
    printf("%d\n", *num);  
}  
  
int *get_num_ptr(void) {  
    int x = 42;  
    return &x;  
}
```

## Solution: get\_num\_ptr.c

## What's wrong with this code?

- Stack use after return bug, `x` is located on the stack which is automatically cleaned when `get_num_ptr()` returns, which invalidates the address

Assume we still want `get_num_ptr()` to return a pointer, how can we fix this code?

- We need to ensure the value lives longer, so we should `malloc` the memory instead, and we can control the lifetime

How did the change made affect each variable's `location` in `memory`?

- The `num` and `x` pointer does not change location as they still live on the stack, however, the value `x` and `num` point to is allocated on the `heap` where `malloc`'d memory is stored

```
○ ○ ○  
  
#include <stdio.h>  
#include <stdlib.h>  
  
int *get_num_ptr(void);  
  
int main(void) {  
    int *num = get_num_ptr();  
  
    printf("%d\n", *num);  
  
    free(num);  
}  
  
int *get_num_ptr(void) {  
    int *x = malloc(sizeof(int));  
    *x = 42;  
  
    return x;  
}
```

## Example:c\_strings.c

What will happen when the above program is compiled and executed?

- When compiled with DCC it will error as str[2] is uninitialized
- With other compilers it will compile without errors
- As **printf expects a string to be null terminated**, so it will keep executing, and str[2] **may often contain '\0'**, or it may **print garbage values**, or even or it may **index outside of the array, and stop with an error**

What does this look like **in memory**?

- char str[10] allocates 10 bytes on the stack, the first two bytes are 'H' and 'i', then 8 bytes of **uninitialised values**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| H | i | ? | ? | ? | ? | ? | ? | ? | ? |
|---|---|---|---|---|---|---|---|---|---|

How could we fix this program?

- We can null terminate the string by setting **str[2] = '\0'**

|   |   |    |   |   |   |   |   |   |   |
|---|---|----|---|---|---|---|---|---|---|
| H | i | \0 | ? | ? | ? | ? | ? | ? | ? |
|---|---|----|---|---|---|---|---|---|---|

○○○

```
#include <stdio.h>
```

```
int main(void) {  
    char str[10];  
    str[0] = 'H';  
    str[1] = 'i';  
    printf("%s", str);  
    return 0;  
}
```

○○○

```
#include <stdio.h>
```

```
int main(void) {  
    char str[10];  
    str[0] = 'H';  
    str[1] = 'i';  
    str[2] = '\0';  
    printf("%s", str);  
    return 0;  
}
```

## Example:c\_strings.c

What will happen when the above program is compiled and executed?

- When compiled with DCC it will error as str[2] is uninitialized
- With other compilers it will compile without errors
- As **printf expects a string to be null terminated**, so it will keep executing, and str[2] **may often contain '\0'**, or it may **print garbage values**, or even or it may **index outside of the array, and stop with an error**

What does this look like **in memory**?

- char str[10] allocates 10 bytes on the stack, the first two bytes are 'H' and 'i', then 8 bytes of **uninitialised values**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| H | i | ? | ? | ? | ? | ? | ? | ? | ? |
|---|---|---|---|---|---|---|---|---|---|

How could we fix this program?

- We can null terminate the string by setting **str[2] = '\0'**

|   |   |    |   |   |   |   |   |   |   |
|---|---|----|---|---|---|---|---|---|---|
| H | i | \0 | ? | ? | ? | ? | ? | ? | ? |
|---|---|----|---|---|---|---|---|---|---|

○○○

```
#include <stdio.h>
```

```
int main(void) {  
    char str[10];  
    str[0] = 'H';  
    str[1] = 'i';  
    printf("%s", str);  
    return 0;  
}
```

○○○

```
#include <stdio.h>
```

```
int main(void) {  
    char str[10];  
    str[0] = 'H';  
    str[1] = 'i';  
    str[2] = '\0';  
    printf("%s", str);  
    return 0;  
}
```



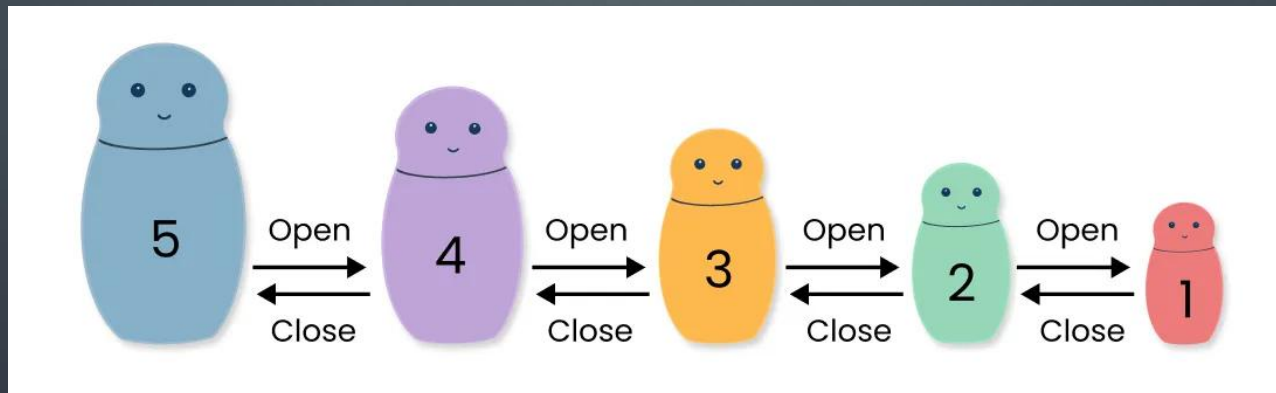
# RECURSION!

## What is recursion?

Recursion is a programming method where a function solves a problem by **calling itself** with **smaller versions of the same problem**. Think of a Russian nesting doll, you open each to find a smaller, identical doll until you reach the final solid one.

To avoid **infinite loops** recursive functions, have two key parts:

1. **The base case:** The simplest possible version of the problem, that doesn't need recursion to be solved, the condition we return a simple answer from
2. **The recursive case:** The part of the function that breaks the problem down into smaller pieces, moving it closer to the base case



## Example: Sum from 1 to n (sum.c)

This function takes in a number n and returns the sum of all integers from 0 to n

### Iterative Version

```
int sum(int n) {  
    int result = 0;  
    for (int i = 0; i <= n; i++) {  
        result += i;  
    }  
    return result;  
}
```

### Recursive Version

```
int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + sum(n-1);  
}
```

What is the difference between the loop and recursive versions?

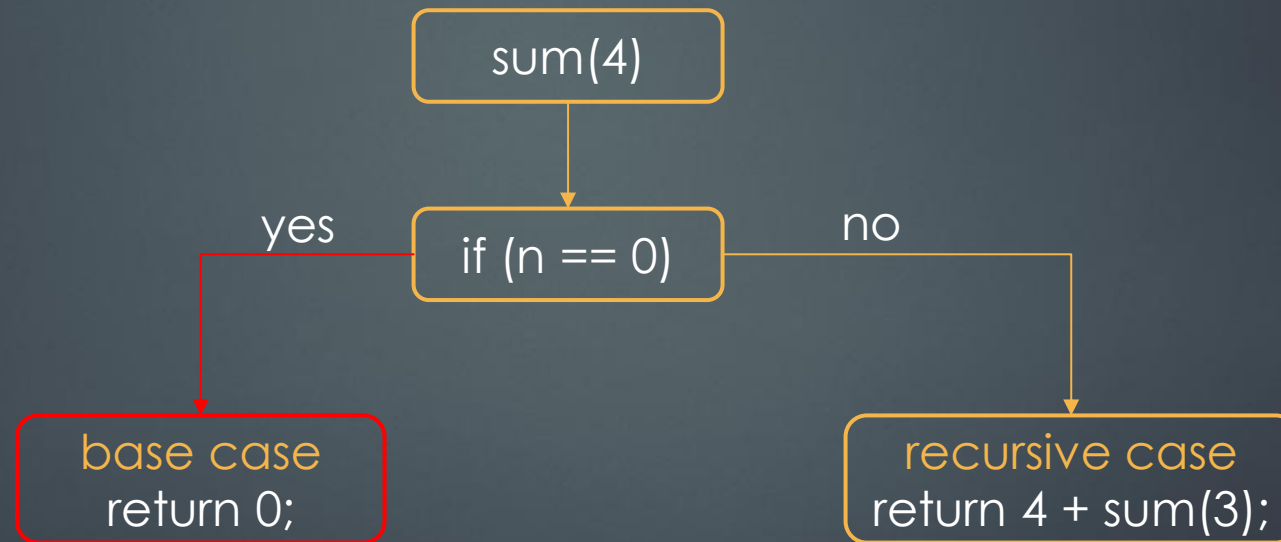
- When a loop is used to calculate the sum there is only one call to the sum function.

What happens in memory when this program runs?

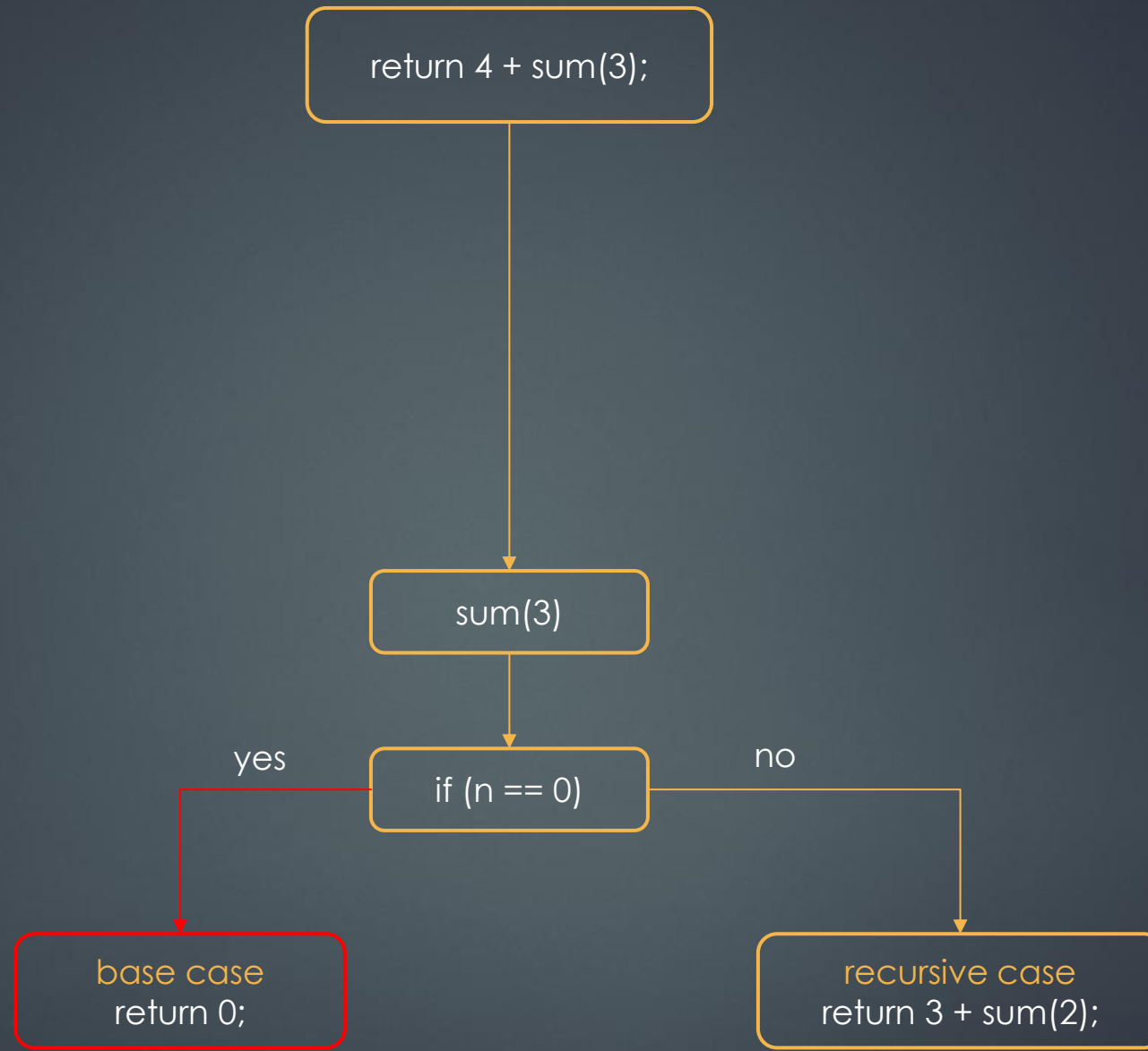
- After a function is called its **arguments and local variables are stored on the stack**.
- In the **recursive version**, there are n calls to the sum function. Each function call **creates a new section of memory (or frame) on the stack** to store its arguments and local variables.

Now we want to call this recursive function with 4, and step through each call to sum()

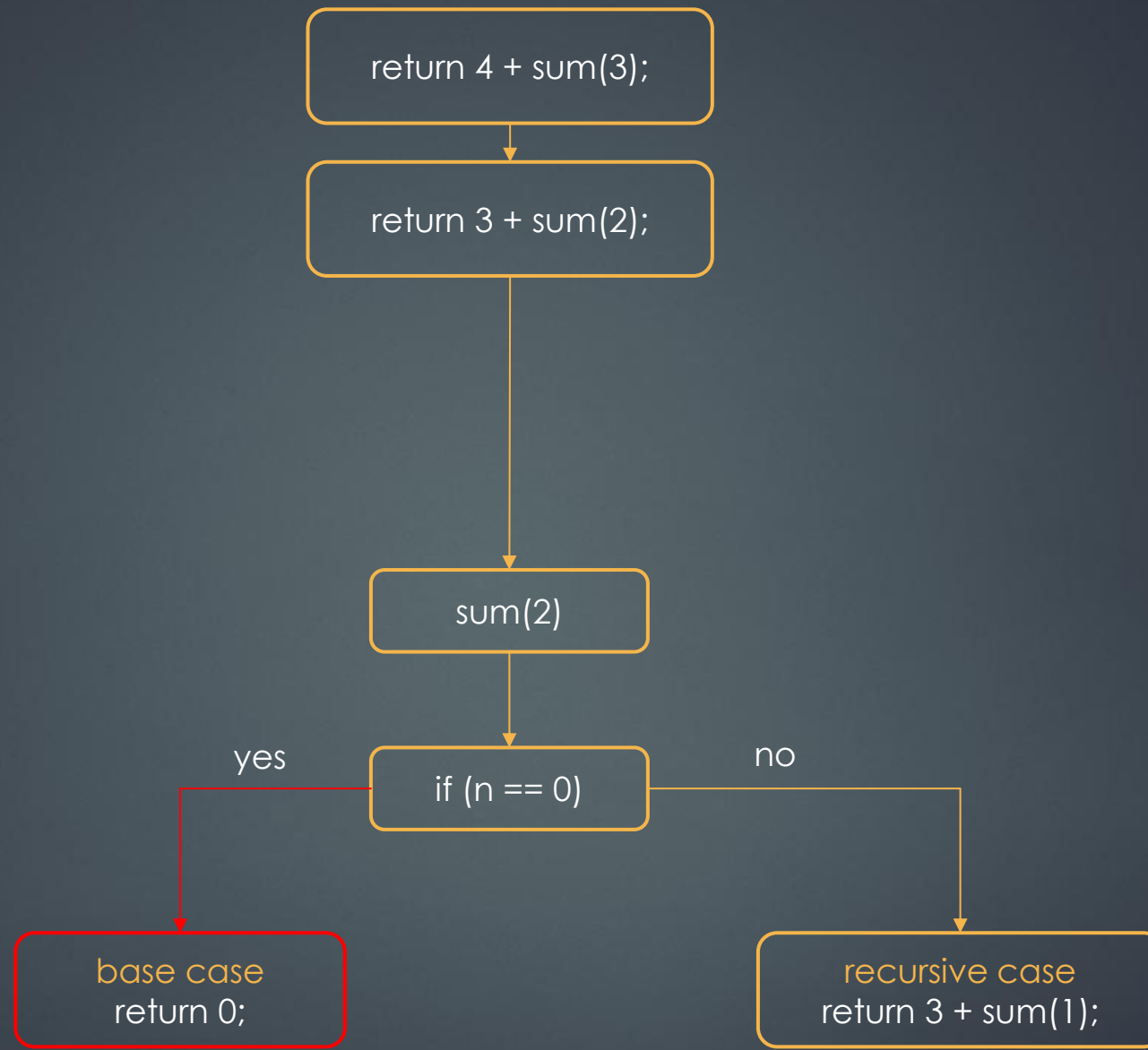
```
○○○  
  
int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + sum(n-1);  
}
```



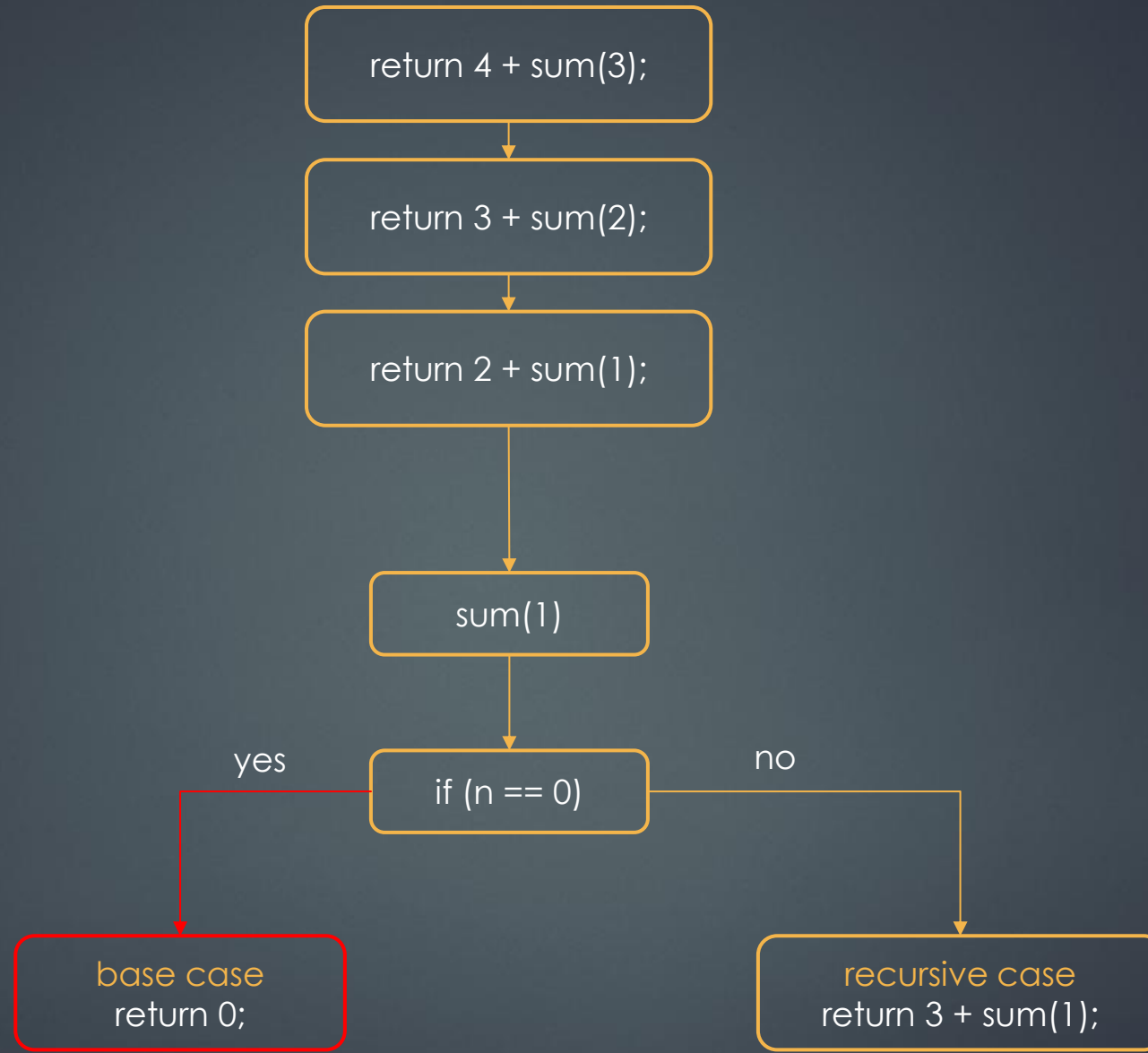
We now repeat this process for each call of sum until we hit the base case of `sum(0)`;



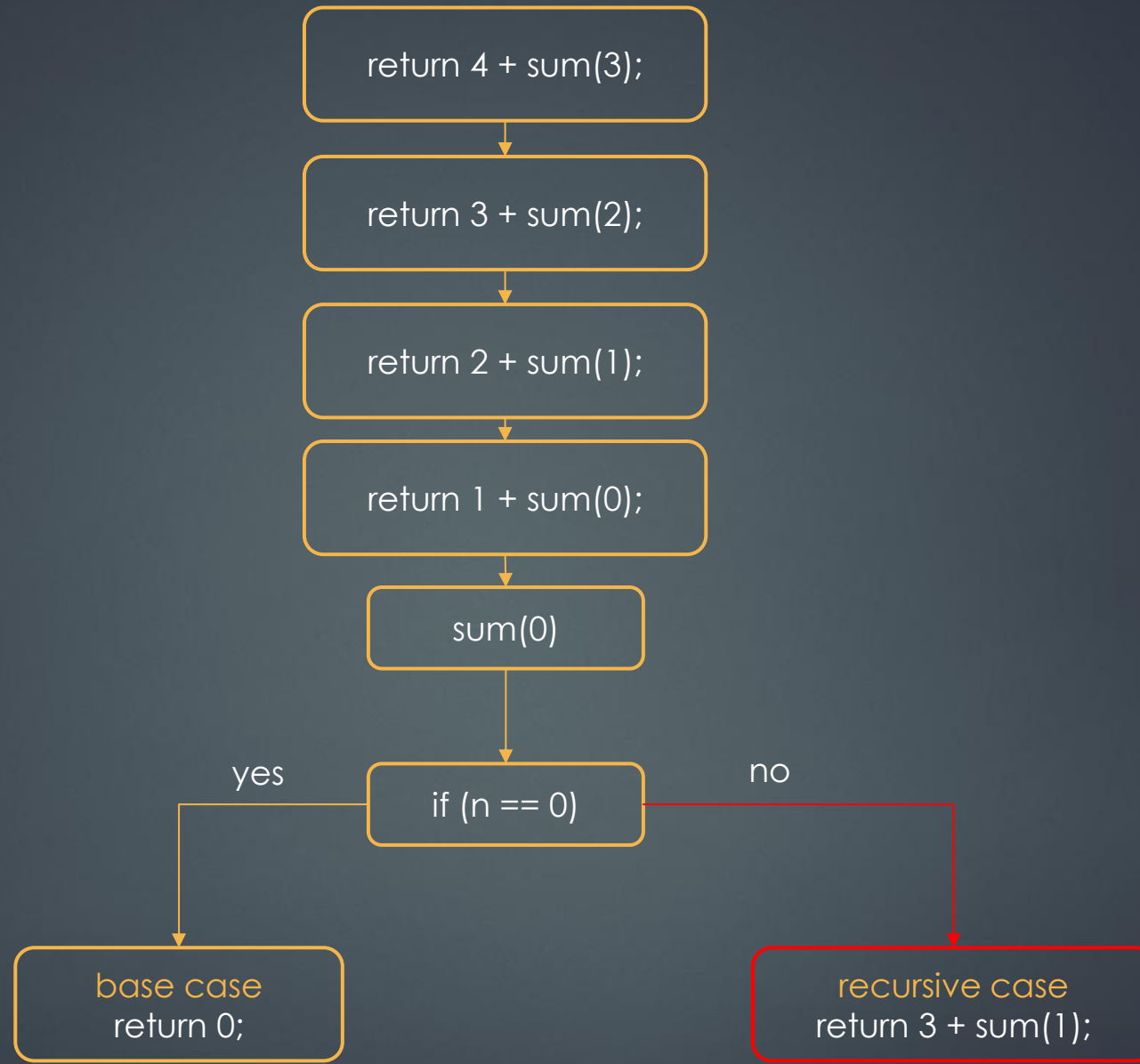
We now repeat this process for each call of sum until we hit the base case of sum(0);



We now repeat this process for each call of `sum` until we hit the base case of `sum(0)`;



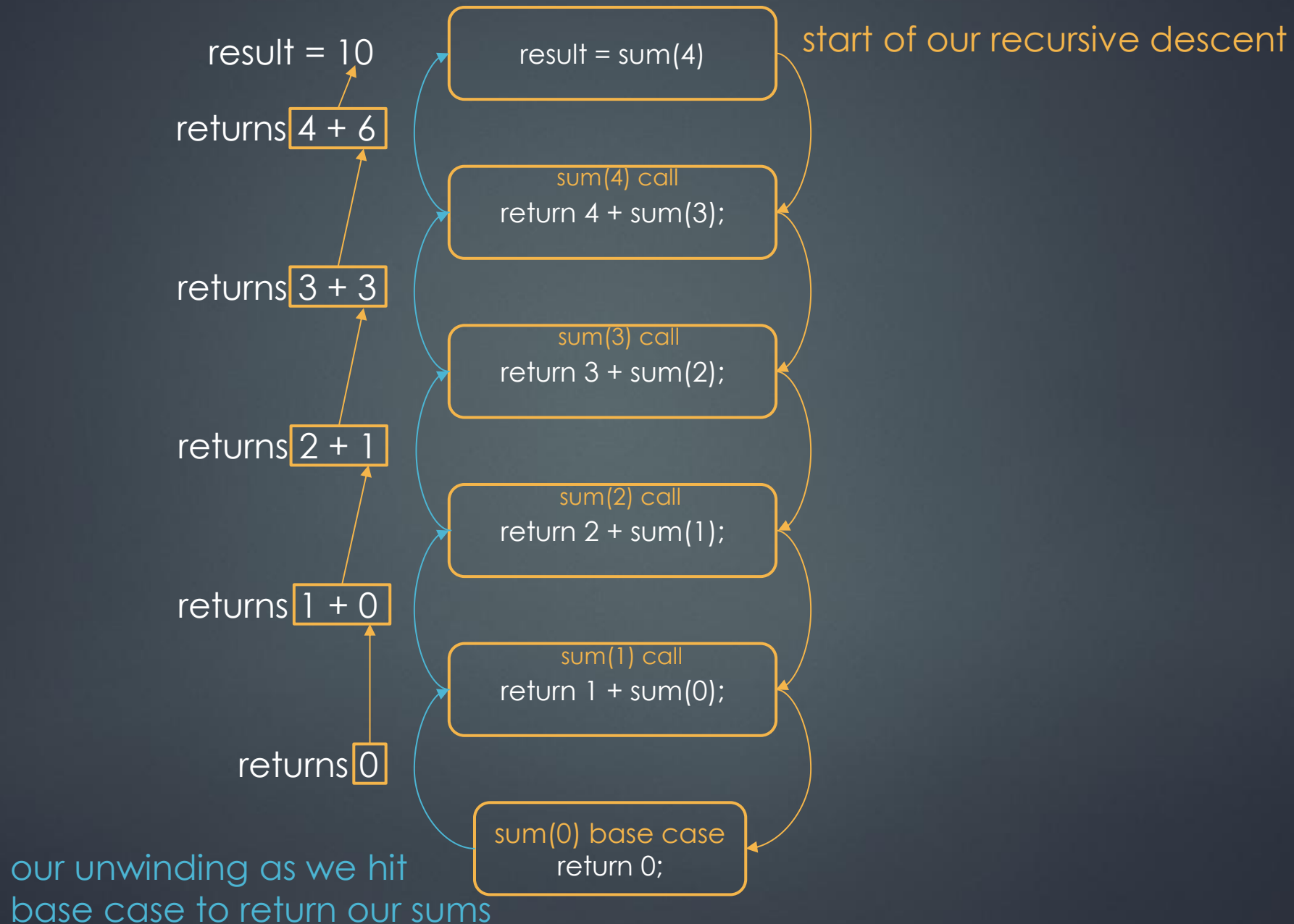
We now repeat this process for each call of `sum` until we hit the base case of `sum(0)`;



We now repeat this process for each call of `sum` until we hit the base case of `sum(0)`;



So now we have hit the base case, and it will return 0, then unwind to the original function call



## Example: Length of Linked List (linked\_list\_\*.c)

### Iterative Version

```
○○○  
  
int length(struct node *head) {  
    int count = 0;  
    struct node *current = head;  
  
    while (current != NULL) {  
        count++;  
        current = current->next;  
    }  
  
    return count;  
}
```

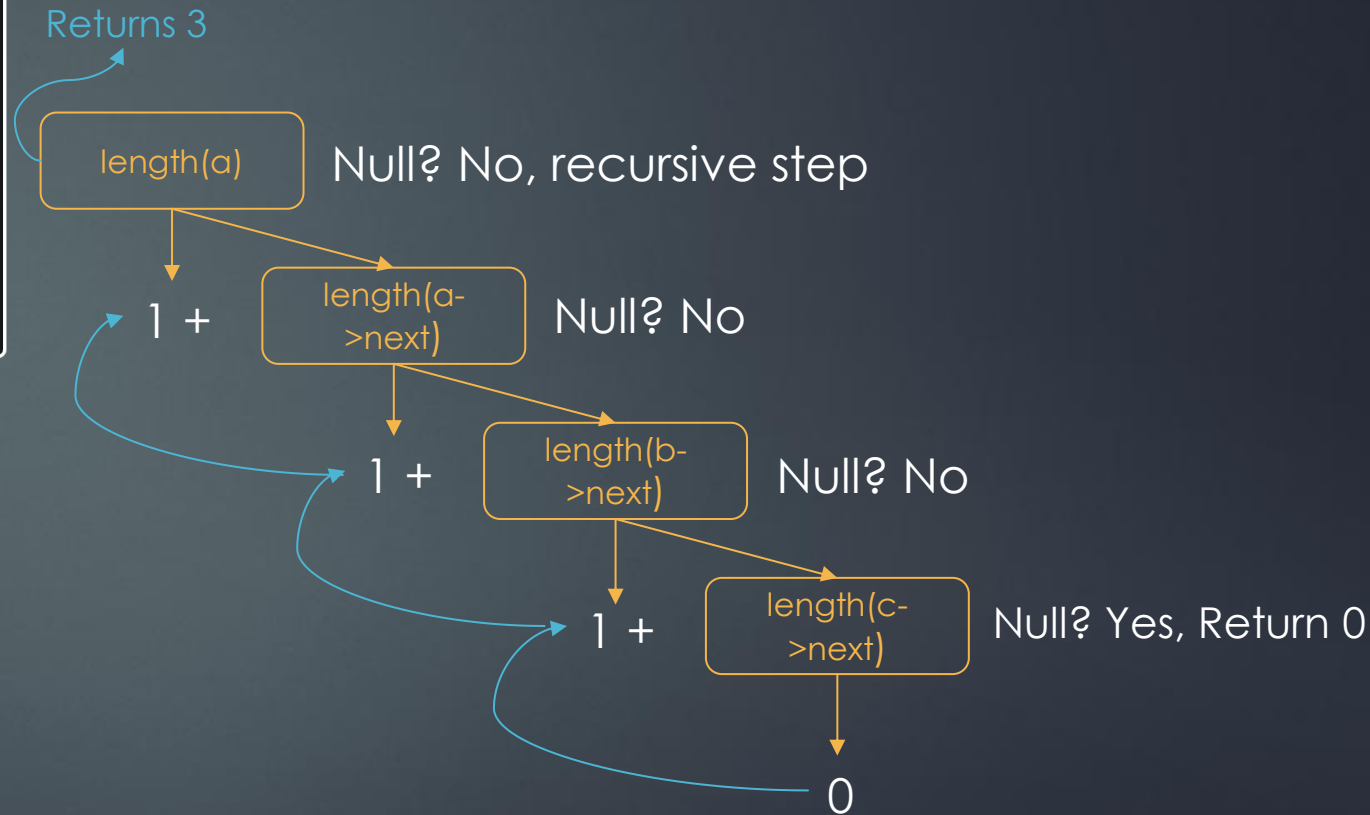
### Recursive Version

```
○○○  
  
int length(struct node *head) {  
    if (head == NULL) {  
        return 0;  
    } else {  
        return 1 + length(head->next);  
    }  
}
```

Example List: {a, b, c}

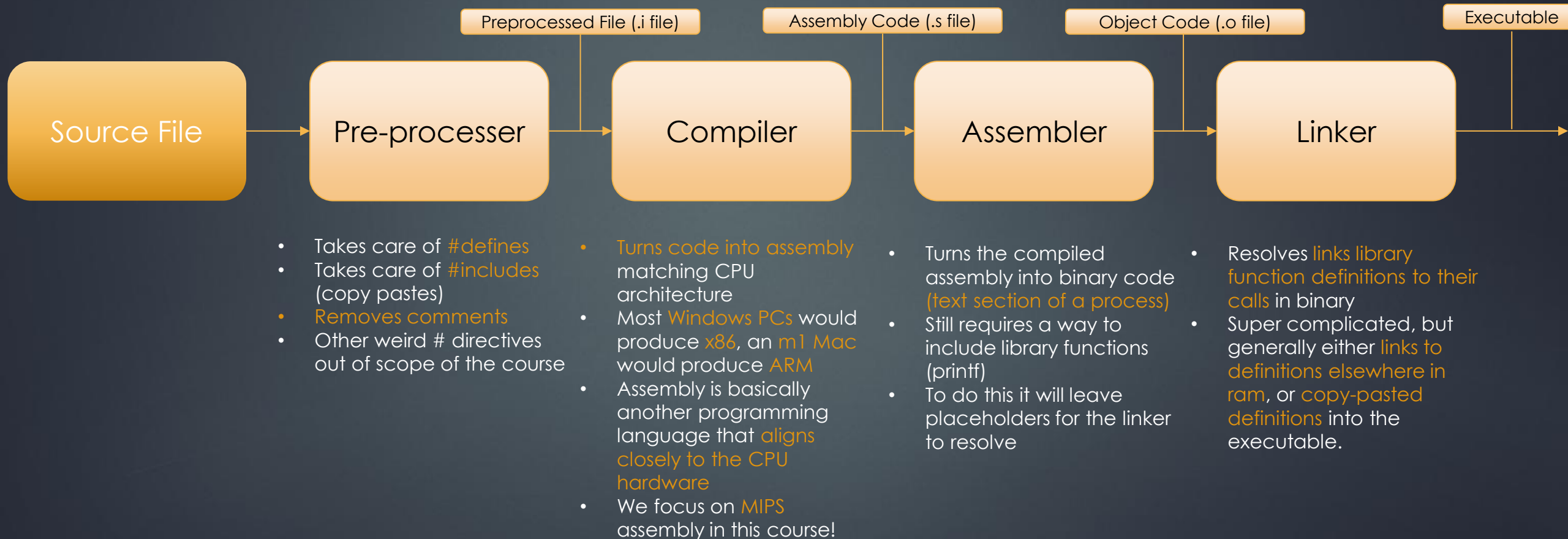


Returns 3



COMPILATION!

# From source to execution



# Producing each step of the compiler

## clang -E x.c

- Executes the C pre-processor and writes modified C code to stdout containing the contents of all #include'd files and replacing all #define'd symbols

## clang -S x.c

- Produces a file x.s containing the assembly code generated by the compiler for the C code in x.c, the type of assembly code is architecture dependent

## clang -c x.c

- Produces a file x.o containing relocatable machine code for the C code in x.c. Also, architecture dependent. This is not a complete program, even if it has a main() function: it needs to be combined with the code for the library functions (by the linker ld)

## clang x.c

- Produces an executable file called a.out, containing all the machine code needed to run the code from x.c on the target machine architecture. The name a.out can be overridden by specifying a flag -o filename.

# ASSEMBLY/MIPS INTRO!

# What is Assembly Code?

As we saw in the compilation, **assembly code is produced by the compiler.**

Think of it like human readable translation of machine code, the lowest level language before you're writing 1's and 0's.

## Comparing assembly to C

**C Code (High-Level):** Say you write a line of code like  $a = b + c$

**Assembly (Low-Level):** This translates into several simple instructions, such as:

- load b into a place
- load c into another place
- add b and c together
- store the result into a

Simply put, assembly is the bridge between your C code and the computer's processor. Assembly's **instructions align directly with the processor's specific architecture.**

# What assembly language do we use? MIPS!

There are many different types of assembly language, one for each kind of CPU architecture (like x86 on your PC or ARM on your phone or MacBook). For this course, we will use **MIPS**.

MIPS is a clean, simple, and elegant architecture that is often used for teaching. Its simplicity makes it perfect for learning the fundamental concepts of how a computer works without the extra complexity of modern commercial processors.

## Our Toolkit: MIPSY

Since your computer probably doesn't run on a MIPS processor, we use an emulator **mipsy**, that lets us run MIPS assembly code on any machine. If you want to get ahead on learning MIPS, this website is the ultimate MIPS resource for COMP1521.

<https://cgi.cse.unsw.edu.au/~cs1521/25T3/resources/mips-guide.html>

Additionally for reference, we can run the MIPS files (\*.s) on cse servers using:

- **1521 mipsy test.s**
- Or we can use mipsy-web, which we will look at more next week!
  - <https://cgi.cse.unsw.edu.au/~cs1521/mipsy/>



# What does a MIPS program look like?

## C Version

```
○○○  
  
int main(void) {  
    int b = 12;  
    int c = 30;  
    printf("%d", b + c);  
}
```

## MIPS Version

```
○○○  
  
.text  
main:  
    li      $t0, 12  
    li      $t1, 30  
  
    add     $t2, $t0, $t1    # Add the two numbers  
  
    li      $v0, 1           # Syscall value for print integer  
  
    move    $a0, $t2         # Move the result to $a0 as the syscall expects it there  
    syscall                               # Run syscall, that currently exists in $v0  
  
    li      $v0, 0           # return 0 from main  
    jr      $ra
```

We'll learn a lot more about the **instructions**, different types of **syscalls** and how to use them all over the next few weeks, it's completely normal if this is a little confusing at first!